

¿Qué hace el sistema y cuáles son sus entidades principales?

El sistema es una aplicación de consola para la gestión de una hacienda ganadera. Permite administrar potreros, el ciclo de vida de las reses, la vacunación, las ventas y la autenticación de usuarios con roles. Las entidades principales son: Hacienda (que actúa como raíz), Potrero, Res y sus subtipos Ternero, Cebon y Novillo, Vacuna y sus subtipos Bacteriana y Viva, Venta, Usuario y la clase Autenticacion, además de una jerarquía de validadores ValidadorRes, ValidadorPotrero, ValidadorVacuna, ValidadorVenta que heredan de la clase abstracta Validacion.

Hacienda implementa tres interfaces: IVacunacion, ICreacionVacuna que define cuatro sobrecargas del método crear_vacuna, y IVentaRes, y concentra al menos cinco responsabilidades: potreros, reses, vacunas, ventas y eventos. Esto hace sospechar de entrada varias violaciones a los principios SOLID: SRP, porque la clase hace demasiado; LSP, porque las subclases de Res lanzan excepciones en el setter de Edad que la clase base no contempla; e ISP, porque cada validador está obligado a implementar cuatro métodos y solo usa uno, lanzando NotImplementedException en los otros tres. La autenticación, por el contrario, esta en una clase aparte y al parecer todos los roles tienen los mismos permisos.

Tres lugares del código que harían costoso un cambio

1. La clase Hacienda (Clases/Hacienda.cs). Concentra cinco responsabilidades en un solo archivo: crear potreros, manejar reses, crear vacunas (cuatro sobrecargas), aplicar vacunas y publicar eventos. Cualquier cambio en cualquiera de esas áreas obliga a tocar el mismo archivo y además instancia clases concretas en vez de abstracciones. Refactorizarla en servicios independientes impactaría la mayoría de los casos de uso.
2. La jerarquía de validadores (Clases/Validaciones/). La clase base Validacion obliga a implementar ValidarRes, ValidarPotrero, ValidarVacuna y ValidarVenta, pero cada validador concreto solo necesita uno y lanza NotImplementedException en los otros tres. Si se agrega una nueva entidad, hay que modificar la base y reescribir los cuatro validadores para que compilen. Segregar una interfaz por contexto IValidarRes, IValidarPotrero, etc. eliminaría esa cascada.
3. La jerarquía de Res (Clases/Res.cs y subclases). La clase base acepta cualquier edad sin lanzar excepciones, pero Ternero, Cebon y Novillo refuerzan precondiciones y lanzan Exception fuera de su rango. Un código que trate a todas las reses como Res puede romper en tiempo de ejecución con una excepción que el contrato base no anunciaba. Como prueba adicional del descuido, Novillo.cs lanza el mensaje El ternero excedió la edad máxima.

Pregunta que le haría al programador original

Cuando modelaste Hacienda como una clase que centraliza potreros, reses, vacunas, ventas y eventos, ¿fue creciendo por acumulación de features? ¿Como penso todo el manejo de roles, usuario y autenticacion?